AIAC Assignment 11.2

Name: M.JAYANTH

HT No: 2303A52485

Batch: 32

Task Description #1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty

methods.

Sample Input Code:

class Stack:

pass

Expected Output:

• A functional stack implementation with all required methods and

docstrings.

```
# Implement a Stack class with push, pop, peek, and is_empty methods. Include
docstrings.
class Stack:
    """A simple implementation of a stack data structure."""

    def __init__(self):
        """Initialize an empty stack."""
        self.items = []

    def push(self, item):
        """Add an item to the top of the stack.

        Args:
            item: The item to be added to the stack.
        """
        self.items.append(item)

    def pop(self):
        """Remove and return the item at the top of the stack.

        Returns:
            The item at the top of the stack.

        Raises:
            IndexError: If the stack is empty.
```

```
        """
        if self.is_empty():
            raise IndexError("Pop from an empty stack")
        return self.items.pop()

    def peek(self):
        """Return the item at the top of the stack without removing it.

        Returns:
            The item at the top of the stack.

        Raises:
            IndexError: If the stack is empty.
        """
        if self.is_empty():
            raise IndexError("Peek from an empty stack")
        return self.items[-1]

    def is_empty(self):
        """Check if the stack is empty.

        Returns:
            True if the stack is empty, False otherwise.
        """
        return len(self.items) == 0
# Example usage:
if __name__ == "__main__":
    stack = Stack()
    stack.push(1)
    stack.push(2)
    stack.push(3)

    print(stack.peek())  # Output: 3
    print(stack.pop())   # Output: 3
    print(stack.is_empty())  # Output: False
    print(stack.pop())   # Output: 2
    print(stack.pop())   # Output: 1
    print(stack.is_empty())  # Output: True
```

=== 1.py ===

3

3

False

2

1

True

Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Sample Input Code:

class Queue:

pass

Expected Output:

• FIFO-based queue class with enqueue, dequeue, peek, and size

methods.

```
# Implement a Queue using Python lists with enqueue, dequeue, peek, and size
methods. Include docstrings.
class Queue:
    """A simple implementation of a queue data structure."""

    def __init__(self):
        """Initialize an empty queue."""
        self.items = []

    def enqueue(self, item):
        """Add an item to the end of the queue.

        Args:
            item: The item to be added to the queue.
        """
        self.items.append(item)

    def dequeue(self):
        """Remove and return the item at the front of the queue.

        Returns:
            The item at the front of the queue.

        Raises:
            IndexError: If the queue is empty.
        """
        if self.is_empty():
            raise IndexError("Dequeue from an empty queue")
        return self.items.pop(0)

    def peek(self):
        """Return the item at the front of the queue without removing it.
```

```
        Returns:
            The item at the front of the queue.

        Raises:
            IndexError: If the queue is empty.
        """
        if self.is_empty():
            raise IndexError("Peek from an empty queue")
        return self.items[0]

    def size(self):
        """Return the number of items in the queue.

        Returns:
            The number of items in the queue.
        """
        return len(self.items)

    def is_empty(self):
        """Check if the queue is empty.

        Returns:
            True if the queue is empty, False otherwise.
        """
        return len(self.items) == 0

# Example usage:
if __name__ == "__main__":
    queue = Queue()
    queue.enqueue(1)
    queue.enqueue(2)
    queue.enqueue(3)

    print(queue.peek())  # Output: 1
    print(queue.dequeue())  # Output: 1
    print(queue.size())  # Output: 2
    print(queue.dequeue())  # Output: 2
    print(queue.dequeue())  # Output: 3
    print(queue.is_empty())  # Output: True
```

=== 2.py ===

1

1

2

2

3

True

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display

methods.

Sample Input Code:

class Node:

pass

class LinkedList:

pass

Expected Output:

• A working linked list implementation with clear method

documentation.

```
# Implement a Singly Linked List with a Node class and a LinkedList class
containing insert
# and display methods. Include docstrings.
class Node:
    """A node in a singly linked list."""

    def __init__(self, data):
        """Initialize a node with the given data and a reference to the next
node.

        Args:
            data: The data stored in the node.
        """
        self.data = data
        self.next = None
class LinkedList:
    """A simple implementation of a singly linked list."""

    def __init__(self):
        """Initialize an empty linked list."""
        self.head = None

    def insert(self, data):
```

```
        """Insert a new node with the given data at the end of the linked
list.

        Args:
            data: The data to be stored in the new node.
        """
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            return
        last_node = self.head
        while last_node.next:
            last_node = last_node.next
        last_node.next = new_node

    def display(self):
        """Display the contents of the linked list."""
        current_node = self.head
        while current_node:
            print(current_node.data, end=" -> ")
            current_node = current_node.next
        print("None")
# Example usage:
if __name__ == "__main__":
    linked_list = LinkedList()
    linked_list.insert(1)
    linked_list.insert(2)
    linked_list.insert(3)

    linked_list.display()  # Output: 1 -> 2 -> 3 -> None
```

=== 3.py ===

1 -> 2 -> 3 -> None

Task Description #4 – Binary Search Tree (BST)

Task: Use AI to create a BST with insert and in-order traversal methods.

Sample Input Code:

class BST:

pass

Expected Output:

• BST implementation with recursive insert and traversal methods.

```python
# Implement a Binary Search Tree (BST) with recursive insert and in-order
traversal methods.
# Include docstrings.
class TreeNode:
    """A node in a binary search tree."""

    def __init__(self, key):
        """Initialize a tree node with the given key and references to left
and right children.

        Args:
            key: The value stored in the node.
        """
        self.key = key
        self.left = None
        self.right = None
class BinarySearchTree:
    """A simple implementation of a binary search tree."""

    def __init__(self):
        """Initialize an empty binary search tree."""
        self.root = None

    def insert(self, key):
        """Insert a new key into the binary search tree.

        Args:
            key: The value to be inserted into the tree.
        """
        if self.root is None:
            self.root = TreeNode(key)
        else:
            self._insert_recursive(self.root, key)

    def _insert_recursive(self, node, key):
        """Helper method to insert a new key into the tree recursively.

        Args:
            node: The current node in the tree.
            key: The value to be inserted.
        """
        if key < node.key:
            if node.left is None:
                node.left = TreeNode(key)
            else:
                self._insert_recursive(node.left, key)
        else:
            if node.right is None:
```

```
                node.right = TreeNode(key)
            else:
                self._insert_recursive(node.right, key)

    def in_order_traversal(self):
        """Perform an in-order traversal of the tree and return a list of
keys.

        Returns:
            A list of keys in in-order traversal order.
        """
        return self._in_order_recursive(self.root)

    def _in_order_recursive(self, node):
        """Helper method to perform in-order traversal recursively.

        Args:
            node: The current node in the tree.

        Returns:
            A list of keys in in-order traversal order.
        """
        keys = []
        if node is not None:
            keys.extend(self._in_order_recursive(node.left))
            keys.append(node.key)
            keys.extend(self._in_order_recursive(node.right))
        return keys
# Example usage:
if __name__ == "__main__":
    bst = BinarySearchTree()
    bst.insert(5)
    bst.insert(3)
    bst.insert(7)
    bst.insert(2)
    bst.insert(4)

    print(bst.in_order_traversal())  # Output: [2, 3, 4, 5, 7]
```

=== 4.py ===

[2, 3, 4, 5, 7]

Task Description #5 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and

delete methods.

Sample Input Code:

class HashTable:

pass

Expected Output:

• Collision handling using chaining, with well-commented methods.

```
# Implement a Hash Table with insert, search, and delete methods
# using chaining for collision handling. Include docstrings.
class HashTable:
    """A simple implementation of a hash table using chaining for collision
handling."""

    def __init__(self, size=10):
        """Initialize the hash table with a specified size.

        Args:
            size: The number of buckets in the hash table (default is 10).
        """
        self.size = size
        self.table = [[] for _ in range(size)]

    def _hash(self, key):
        """Compute the hash value for a given key.

        Args:
            key: The key to be hashed.

        Returns:
            The hash value corresponding to the key.
        """
        return hash(key) % self.size

    def insert(self, key, value):
        """Insert a key-value pair into the hash table.

        Args:
            key: The key to be inserted.
            value: The value associated with the key.
        """
        index = self._hash(key)
        bucket = self.table[index]
        for i, (k, v) in enumerate(bucket):
            if k == key:
                bucket[i] = (key, value)  # Update existing key
                return
        bucket.append((key, value))  # Insert new key-value pair
```

```
    def search(self, key):
        """Search for a value associated with a given key.

        Args:
            key: The key to be searched.

        Returns:
            The value associated with the key if found, otherwise None.
        """
        index = self._hash(key)
        bucket = self.table[index]
        for k, v in bucket:
            if k == key:
                return v
        return None  # Key not found

    def delete(self, key):
        """Delete a key-value pair from the hash table.

        Args:
            key: The key to be deleted.
        """
        index = self._hash(key)
        bucket = self.table[index]
        for i, (k, v) in enumerate(bucket):
            if k == key:
                del bucket[i]  # Remove the key-value pair
                return
# Example usage:
if __name__ == "__main__":
    hash_table = HashTable()
    hash_table.insert("name", "Alice")
    hash_table.insert("age", 30)
    hash_table.insert("city", "New York")

    print(hash_table.search("name"))  # Output: Alice
    print(hash_table.search("age"))   # Output: 30
    print(hash_table.search("city"))  # Output: New York

    hash_table.delete("age")
    print(hash_table.search("age"))   # Output: None
```

=== 5.py ===

Alice

30

New York

None

Task Description #6 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code:

class Graph:

pass

Expected Output:

• Graph with methods to add vertices, add edges, and display

connections.

```
# Implement a Graph using an adjacency list with methods to add_vertex,
# add_edge, and display connections. Include docstrings.
class Graph:
    """A simple implementation of a graph using an adjacency list."""

    def __init__(self):
        """Initialize an empty graph."""
        self.adjacency_list = {}

    def add_vertex(self, vertex):
        """Add a vertex to the graph.

        Args:
            vertex: The vertex to be added to the graph.
        """
        if vertex not in self.adjacency_list:
            self.adjacency_list[vertex] = []

    def add_edge(self, vertex1, vertex2):
        """Add an edge between two vertices in the graph.

        Args:
            vertex1: The first vertex.
            vertex2: The second vertex.
        """
        if vertex1 in self.adjacency_list and vertex2 in self.adjacency_list:
            self.adjacency_list[vertex1].append(vertex2)
            self.adjacency_list[vertex2].append(vertex1)

    def display(self):
        """Display the connections in the graph."""
```

```
        for vertex, edges in self.adjacency_list.items():
            print(f"{vertex}: {', '.join(edges)}")
# Example usage:
if __name__ == "__main__":
    graph = Graph()
    graph.add_vertex("A")
    graph.add_vertex("B")
    graph.add_vertex("C")

    graph.add_edge("A", "B")
    graph.add_edge("A", "C")

    graph.display()  # Output: A: B, C; B: A; C: A
```

=== 6.py ===

A: B, C

B: A

C: A

Task Description #7 – Priority Queue

Task: Use AI to implement a priority queue using Python's heapq

module.

Sample Input Code:

class PriorityQueue:

pass

Expected Output:

• Implementation with enqueue (priority), dequeue (highest priority),

and display methods.

```
# Implement a Priority Queue using Python's heapq module with enqueue,
# dequeue, and display methods. Include docstrings.
import heapq
class PriorityQueue:
    """A simple implementation of a priority queue using Python's heapq
module."""

    def __init__(self):
        """Initialize an empty priority queue."""
        self.elements = []
```

```
    def enqueue(self, item, priority):
        """Add an item to the priority queue with a given priority.

        Args:
            item: The item to be added to the priority queue.
            priority: The priority of the item (lower values indicate higher
priority).
        """
        heapq.heappush(self.elements, (priority, item))

    def dequeue(self):
        """Remove and return the item with the highest priority from the
queue.

        Returns:
            The item with the highest priority.

        Raises:
            IndexError: If the priority queue is empty.
        """
        if self.is_empty():
            raise IndexError("Dequeue from an empty priority queue")
        return heapq.heappop(self.elements)[1]

    def display(self):
        """Display the contents of the priority queue."""
        for priority, item in sorted(self.elements):
            print(f"Priority: {priority}, Item: {item}")

    def is_empty(self):
        """Check if the priority queue is empty.

        Returns:
            True if the priority queue is empty, False otherwise.
        """
        return len(self.elements) == 0
# Example usage:
if __name__ == "__main__":
    pq = PriorityQueue()
    pq.enqueue("Task 1", priority=2)
    pq.enqueue("Task 2", priority=1)
    pq.enqueue("Task 3", priority=3)

    pq.display()  # Output: Priority: 1, Item: Task 2; Priority: 2, Item: Task
1; Priority: 3, Item: Task 3

    print(pq.dequeue())  # Output: Task 2
```

```
    print(pq.dequeue())  # Output: Task 1
    print(pq.is_empty())  # Output: False
    print(pq.dequeue())  # Output: Task 3
    print(pq.is_empty())  # Output: True
```

=== 7.py ===

Priority: 1, Item: Task 2

Priority: 2, Item: Task 1

Priority: 3, Item: Task 3

Task 2

Task 1

False

Task 3

True

Task Description #8 – Deque

Task: Use AI to implement a double-ended queue using

collections.deque.

Sample Input Code:

class DequeDS:

pass

Expected Output:

• Insert and remove from both ends with docstrings.

```
# Implement a Deque (double-ended queue) using collections.
# deque with insert and remove methods for both ends. Include docstrings.
from collections import deque
class Deque:
    """A simple implementation of a double-ended queue (deque) using
collections.deque."""

    def __init__(self):
        """Initialize an empty deque."""
        self.deque = deque()

    def insert_front(self, item):
        """Insert an item at the front of the deque.
```

```
        Args:
            item: The item to be added to the front of the deque.
        """
        self.deque.appendleft(item)

    def insert_rear(self, item):
        """Insert an item at the rear of the deque.

        Args:
            item: The item to be added to the rear of the deque.
        """
        self.deque.append(item)

    def remove_front(self):
        """Remove and return the item at the front of the deque.

        Returns:
            The item at the front of the deque.

        Raises:
            IndexError: If the deque is empty.
        """
        if self.is_empty():
            raise IndexError("Remove from an empty deque")
        return self.deque.popleft()

    def remove_rear(self):
        """Remove and return the item at the rear of the deque.

        Returns:
            The item at the rear of the deque.

        Raises:
            IndexError: If the deque is empty.
        """
        if self.is_empty():
            raise IndexError("Remove from an empty deque")
        return self.deque.pop()

    def is_empty(self):
        """Check if the deque is empty.

        Returns:
            True if the deque is empty, False otherwise.
        """
        return len(self.deque) == 0
# Example usage:
if __name__ == "__main__":
```

```
    my_deque = Deque()
    my_deque.insert_rear(1)
    my_deque.insert_rear(2)
    my_deque.insert_front(0)

    print(my_deque.remove_front())  # Output: 0
    print(my_deque.remove_rear())   # Output: 2
    print(my_deque.is_empty())      # Output: False
    print(my_deque.remove_front())  # Output: 1
    print(my_deque.is_empty())      # Output: True
```

=== 8.py ===

0

2

False

1

True

Task Description #9 Real-Time Application Challenge – Choose the

Right Data Structure

Scenario:

Your college wants to develop a Campus Resource Management System

that handles:

1. Student Attendance Tracking – Daily log of students

entering/exiting the campus.

2. Event Registration System – Manage participants in events with

quick search and removal.

3. Library Book Borrowing – Keep track of available books and their

due dates.

4. Bus Scheduling System – Maintain bus routes and stop

connections.

5. Cafeteria Order Queue – Serve students in the order they arrive.

Student Task:

• For each feature, select the most appropriate data structure from

the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with

AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature

with comments and docstrings.

9.1

```
"""
Implement a Hash Table for Student Attendance Tracking.
It should log students entering and exiting the campus using their Student ID
for quick O(1) lookups.
Include methods to mark_present, mark_absent, and check_status.
"""
class StudentAttendance:
    def __init__(self):
        self.attendance = {}

    def mark_present(self, student_id):
        """Mark a student as present"""
        self.attendance[student_id] = "present"

    def mark_absent(self, student_id):
        """Mark a student as absent"""
        self.attendance[student_id] = "absent"

    def check_status(self, student_id):
        """Check attendance status of a student"""
        return self.attendance.get(student_id, "not recorded")
```

```
# Example usage:
if __name__ == "__main__":
    attendance_tracker = StudentAttendance()
    attendance_tracker.mark_present("S123")
    attendance_tracker.mark_absent("S124")

    print(attendance_tracker.check_status("S123"))  # Output: present
    print(attendance_tracker.check_status("S124"))  # Output: absent
    print(attendance_tracker.check_status("S125"))  # Output: not recorded
```

9.2

```
"""
Implement a Binary Search Tree (BST) for an Event Registration System.
It should manage event participants by their ID to allow quick search,
insertion, and removal.
Include methods to register_participant, remove_participant, and
find_participant.
"""
class Node:
    def __init__(self, participant_id):
        self.participant_id = participant_id
        self.left = None
        self.right = None

class BinarySearchTree:
    def __init__(self):
        self.root = None

    def register_participant(self, participant_id):
        # Implementation for registering a participant
        if self.root is None:
            self.root = Node(participant_id)
        else:
            self._insert_recursive(self.root, participant_id)

    def _insert_recursive(self, node, participant_id):
        if participant_id < node.participant_id:
            if node.left is None:
                node.left = Node(participant_id)
            else:
                self._insert_recursive(node.left, participant_id)
        else:
            if node.right is None:
                node.right = Node(participant_id)
            else:
                self._insert_recursive(node.right, participant_id)
```

```
    def remove_participant(self, participant_id):
        # Implementation for removing a participant
        self.root = self._remove_recursive(self.root, participant_id)

    def _remove_recursive(self, node, participant_id):
        if node is None:
            return None
        if participant_id < node.participant_id:
            node.left = self._remove_recursive(node.left, participant_id)
        elif participant_id > node.participant_id:
            node.right = self._remove_recursive(node.right, participant_id)
        else:
            if node.left is None:
                return node.right
            elif node.right is None:
                return node.left
            min_larger_node = self._find_min(node.right)
            node.participant_id = min_larger_node.participant_id
            node.right = self._remove_recursive(node.right,
min_larger_node.participant_id)
        return node

    def _find_min(self, node):
        while node.left is not None:
            node = node.left
        return node

    def find_participant(self, participant_id):
        # Implementation for finding a participant
        return self._search_recursive(self.root, participant_id)

    def _search_recursive(self, node, participant_id):
        if node is None:
            return False
        if participant_id == node.participant_id:
            return True
        elif participant_id < node.participant_id:
            return self._search_recursive(node.left, participant_id)
        else:
            return self._search_recursive(node.right, participant_id)
# Example usage:
if __name__ == "__main__":
    event_registration = BinarySearchTree()
    event_registration.register_participant("P001")
    event_registration.register_participant("P002")

    print(event_registration.find_participant("P001"))  # Output: True
    print(event_registration.find_participant("P003"))  # Output: False
```

```
    event_registration.remove_participant("P001")
    print(event_registration.find_participant("P001"))  # Output: False
```

9.3

```
"""
Implement a Hash Table for Library Book Borrowing.
It should keep track of available books, who borrowed them, and their due
dates using the Book ID as the key.
Include methods to borrow_book, return_book, and check_availability.
"""
class Library:
    def __init__(self):
        self.books = {}

    def borrow_book(self, book_id, borrower_name, due_date):
        """Borrow a book from the library"""
        if book_id in self.books and self.books[book_id]['status'] ==
'available':
            self.books[book_id] = {
                'status': 'borrowed',
                'borrower': borrower_name,
                'due_date': due_date
            }
            return f"Book {book_id} borrowed by {borrower_name} until
{due_date}."
        elif book_id in self.books:
            return f"Book {book_id} is currently borrowed by
{self.books[book_id]['borrower']}."
        else:
            return f"Book {book_id} does not exist in the library."

    def return_book(self, book_id):
        """Return a book to the library"""
        if book_id in self.books and self.books[book_id]['status'] ==
'borrowed':
            self.books[book_id] = {'status': 'available'}
            return f"Book {book_id} has been returned and is now available."
        elif book_id in self.books:
            return f"Book {book_id} is already available."
        else:
            return f"Book {book_id} does not exist in the library."

    def check_availability(self, book_id):
        """Check if a book is available for borrowing"""
        if book_id in self.books:
```

```
            return f"Book {book_id} is {self.books[book_id]['status']}."
        else:
            return f"Book {book_id} does not exist in the library."
# Example usage:
if __name__ == "__main__":
    library = Library()
    library.books = {
        "B001": {'status': 'available'},
        "B002": {'status': 'available'},
        "B003": {'status': 'available'}
    }

    print(library.borrow_book("B001", "Alice", "2024-07-01"))  # Book B001
borrowed by Alice until 2024-07-01.
    print(library.check_availability("B001"))  # Book B001 is borrowed.
    print(library.return_book("B001"))  # Book B001 has been returned and is
now available.
    print(library.check_availability("B001"))  # Book B001 is available.
    print(library.borrow_book("B002", "Bob", "2024-07-15"))  # Book B002
borrowed by Bob until 2024-07-15.
    print(library.borrow_book("B002", "Charlie", "2024-07-20"))  # Book B002
is currently borrowed by Bob.
    print(library.check_availability("B002"))  # Book B002 is borrowed.
```

9.4

```
"""
Implement a Graph using an adjacency list for a Bus Scheduling System.
It should maintain bus routes by connecting different bus stops (vertices)
with routes (edges).
Include methods to add_stop, add_route, and display_routes.
"""
class Graph:
    """A simple implementation of a graph using an adjacency list for a bus
scheduling system."""

    def __init__(self):
        """Initialize an empty graph."""
        self.adjacency_list = {}

    def add_stop(self, stop):
        """Add a bus stop to the graph.

        Args:
            stop: The bus stop to be added to the graph.
        """
        if stop not in self.adjacency_list:
            self.adjacency_list[stop] = []
```

```
    def add_route(self, stop1, stop2):
        """Add a route between two bus stops in the graph.

        Args:
            stop1: The first bus stop.
            stop2: The second bus stop.
        """
        if stop1 in self.adjacency_list and stop2 in self.adjacency_list:
            self.adjacency_list[stop1].append(stop2)
            self.adjacency_list[stop2].append(stop1)

    def display_routes(self):
        """Display the routes between bus stops in the graph."""
        for stop, routes in self.adjacency_list.items():
            print(f"{stop}: {', '.join(routes)}")
# Example usage:
if __name__ == "__main__":
    bus_graph = Graph()
    bus_graph.add_stop("Stop A")
    bus_graph.add_stop("Stop B")
    bus_graph.add_stop("Stop C")

    bus_graph.add_route("Stop A", "Stop B")
    bus_graph.add_route("Stop A", "Stop C")

    bus_graph.display_routes()  # Output: Stop A: Stop B, Stop C; Stop B: Stop
A; Stop C: Stop As
```

9.5

```
"""
Implement a Queue for a Cafeteria Order System.
It should serve students strictly in the order they arrive (FIFO).
Include methods to add_order, serve_order, and view_pending_orders.
"""
class CafeteriaQueue:
    def __init__(self):
        self.queue = []

    def add_order(self, order):
        """Add an order to the queue."""
        self.queue.append(order)

    def serve_order(self):
        """Serve the next order in the queue."""
        if self.queue:
            return self.queue.pop(0)
```

```
        else:
            return "No orders to serve."

    def view_pending_orders(self):
        """View all pending orders in the queue."""
        return self.queue
# Example usage:
if __name__ == "__main__":
    cafeteria_queue = CafeteriaQueue()
    cafeteria_queue.add_order("Order 1: Sandwich")
    cafeteria_queue.add_order("Order 2: Salad")
    cafeteria_queue.add_order("Order 3: Soup")

    print(cafeteria_queue.view_pending_orders())  # Output: ['Order 1:
Sandwich', 'Order 2: Salad', 'Order 3: Soup']
    print(cafeteria_queue.serve_order())          # Output: 'Order 1:
Sandwich'
    print(cafeteria_queue.view_pending_orders())  # Output: ['Order 2: Salad',
'Order 3: Soup']
```

=== 9.1.py ===

present

absent

not recorded

=== 9.2.py ===

True

False

False

=== 9.3.py ===

Book B001 borrowed by Alice until 2024-07-01.

Book B001 is borrowed.

Book B001 has been returned and is now available.

Book B001 is available.

Book B002 borrowed by Bob until 2024-07-15.

Book B002 is currently borrowed by Bob.

Book B002 is borrowed.

=== 9.4.py ===

Stop A: Stop B, Stop C

Stop B: Stop A

Stop C: Stop A

=== 9.5.py ===

['Order 1: Sandwich', 'Order 2: Salad', 'Order 3: Soup']

Order 1: Sandwich

['Order 2: Salad', 'Order 3: Soup']

Feature	Data Structure	Justification
Student Attendance Tracking	Hash Table	O(1) lookup by student ID for instant mark/check.
Event Registration System	BST	Fast insert, search, and delete of participant IDs in sorted order.
Library Book Borrowing	Hash Table	Instant retrieval of book status and borrower info by Book ID.
Bus Scheduling System	Graph	Naturally models stops (vertices) and routes (edges) between them.
Cafeteria Order Queue	Queue	FIFO ensures students are served strictly in arrival order.

Task Description #10: Smart Hospital Management System – Data

Structure Selection

A hospital wants to develop a Smart Hospital Management System that

handles:

1. Patient Check-In System – Patients are registered and treated in

order of arrival.

2. Emergency Case Handling – Critical patients must be treated first.

3. Medical Records Storage – Fast retrieval of patient details using

ID.

4. Doctor Appointment Scheduling – Appointments sorted by time.

5. Hospital Room Navigation – Represent connections between

wards and rooms.

Student Task

• For each feature, select the most appropriate data structure from

the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with

AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature

with comments and docstrings.

10.1

```
"""
Implement a Queue for a Patient Check-In System.
It should register and treat patients in the exact order of their arrival
(FIFO).
Include methods to add_patient, call_next_patient, and view_waiting_list.
"""
class PatientQueue:
    def __init__(self):
        self.queue = []

    def add_patient(self, patient_name):
        """Add a patient to the queue."""
        self.queue.append(patient_name)
```

```
    def call_next_patient(self):
        """Call the next patient in the queue."""
        if self.queue:
            return self.queue.pop(0)
        else:
            return "No patients to call."

    def view_waiting_list(self):
        """View all patients currently waiting in the queue."""
        return self.queue
# Example usage:
if __name__ == "__main__":
    patient_queue = PatientQueue()
    patient_queue.add_patient("Patient A")
    patient_queue.add_patient("Patient B")
    patient_queue.add_patient("Patient C")

    print(patient_queue.view_waiting_list())  # Output: ['Patient A', 'Patient
B', 'Patient C']
    print(patient_queue.call_next_patient())   # Output: 'Patient A'
    print(patient_queue.view_waiting_list())  # Output: ['Patient B', 'Patient
C']
```

10.2

```
"""
Implement a Priority Queue for Emergency Case Handling.
It should ensure critical patients are treated before routine cases.
Include methods to admit_patient(name, severity), treat_highest_priority, and
view_queue.
"""
import heapq
class EmergencyQueue:
    def __init__(self):
        self.queue = []

    def admit_patient(self, name, severity):
        """Admit a patient with a given severity level."""
        heapq.heappush(self.queue, (severity, name))

    def treat_highest_priority(self):
        """Treat the patient with the highest priority (lowest severity)."""
        if self.queue:
            return heapq.heappop(self.queue)[1]
        else:
            return "No patients to treat."

    def view_queue(self):
        """View all patients currently in the queue sorted by severity."""
        return sorted(self.queue)
```

```
# Example usage:
if __name__ == "__main__":
    emergency_queue = EmergencyQueue()
    emergency_queue.admit_patient("Patient A", severity=2)
    emergency_queue.admit_patient("Patient B", severity=1)
    emergency_queue.admit_patient("Patient C", severity=3)

    print(emergency_queue.view_queue())  # Output: [(1, 'Patient B'), (2,
'Patient A'), (3, 'Patient C')]
    print(emergency_queue.treat_highest_priority())  # Output: 'Patient B'
    print(emergency_queue.view_queue())  # Output: [(2, 'Patient A'), (3,
'Patient C')]
```

```
10.3"""

Implement a Hash Table for Medical Records Storage.
It should allow fast retrieval of patient details using their Patient ID as
the key.
Include methods to add_record, get_record, and delete_record.
"""
class MedicalRecordsHashTable:
    def __init__(self, size=100):
        self.size = size
        self.table = [[] for _ in range(size)]

    def _hash(self, patient_id):
        return hash(patient_id) % self.size

    def add_record(self, patient_id, details):
        """Add a patient record with ID and details."""
        index = self._hash(patient_id)
        for i, (pid, record) in enumerate(self.table[index]):
            if pid == patient_id:
                self.table[index][i] = (patient_id, details)
                return
        self.table[index].append((patient_id, details))

    def get_record(self, patient_id):
        """Retrieve patient details by Patient ID."""
        index = self._hash(patient_id)
        for pid, details in self.table[index]:
            if pid == patient_id:
                return details
        return None

    def delete_record(self, patient_id):
        """Delete a patient record by Patient ID."""
        index = self._hash(patient_id)
```

```
        self.table[index] = [(pid, details) for pid, details in
self.table[index]
                             if pid != patient_id]
# Example usage:
if __name__ == "__main__":
    medical_records = MedicalRecordsHashTable()
    medical_records.add_record("P001", {"name": "John Doe", "age": 30,
"condition": "Flu"})
    medical_records.add_record("P002", {"name": "Jane Smith", "age": 25,
"condition": "Cold"})

    print(medical_records.get_record("P001"))  # Output: {'name': 'John Doe',
'age': 30, 'condition': 'Flu'}
    print(medical_records.get_record("P002"))  # Output: {'name': 'Jane
Smith', 'age': 25, 'condition': 'Cold'}

    medical_records.delete_record("P001")
    print(medical_records.get_record("P001"))  # Output: None
```

10.4

```
"""
Implement a Binary Search Tree (BST) for Doctor Appointment Scheduling.
It should sort appointments by time to easily spot conflicts and keep a
schedule.
Include methods to schedule_appointment, cancel_appointment, and
view_schedule_in_order.
"""
class AppointmentNode:
    def __init__(self, time, patient_name):
        self.time = time
        self.patient_name = patient_name
        self.left = None
        self.right = None
class AppointmentBST:
    def __init__(self):
        self.root = None

    def schedule_appointment(self, time, patient_name):
        """Schedule a new appointment in the BST.

        Args:
            time: The time of the appointment (e.g., "10:00 AM").
            patient_name: The name of the patient for the appointment.
        """
```

```
        if self.root is None:
            self.root = AppointmentNode(time, patient_name)
        else:
            self._insert_recursive(self.root, time, patient_name)

    def _insert_recursive(self, node, time, patient_name):
        if time < node.time:
            if node.left is None:
                node.left = AppointmentNode(time, patient_name)
            else:
                self._insert_recursive(node.left, time, patient_name)
        else:
            if node.right is None:
                node.right = AppointmentNode(time, patient_name)
            else:
                self._insert_recursive(node.right, time, patient_name)

    def cancel_appointment(self, time):
        """Cancel an appointment at a specific time.

        Args:
            time: The time of the appointment to be canceled.
        """
        self.root = self._remove_recursive(self.root, time)

    def _remove_recursive(self, node, time):
        if node is None:
            return None
        if time < node.time:
            node.left = self._remove_recursive(node.left, time)
        elif time > node.time:
            node.right = self._remove_recursive(node.right, time)
        else:
            if node.left is None:
                return node.right
            elif node.right is None:
                return node.left
            min_larger_node = self._find_min(node.right)
            node.time = min_larger_node.time
            node.patient_name = min_larger_node.patient_name
            node.right = self._remove_recursive(node.right,
min_larger_node.time)
        return node

    def view_schedule_in_order(self):
        """View all scheduled appointments in order of their times."""
        return self._in_order_recursive(self.root)
```

```
    def _in_order_recursive(self, node):
        appointments = []
        if node is not None:
            appointments.extend(self._in_order_recursive(node.left))
            appointments.append((node.time, node.patient_name))
            appointments.extend(self._in_order_recursive(node.right))
        return appointments
# Example usage:
if __name__ == "__main__":
    schedule = AppointmentBST()
    schedule.schedule_appointment("10:00 AM", "Alice")
    schedule.schedule_appointment("11:00 AM", "Bob")
    schedule.schedule_appointment("9:00 AM", "Charlie")
    print("Scheduled Appointments (In Order):",
schedule.view_schedule_in_order())
    schedule.cancel_appointment("10:00 AM")
    print("Scheduled Appointments after cancellation (In Order):",
schedule.view_schedule_in_order())
```

10.5

```
"""
Implement an undirected Graph for Hospital Room Navigation.
It should represent physical connections between different wards and rooms.
Include methods to add_room, connect_rooms, and show_connections.
"""
class Graph:
    """A simple implementation of an undirected graph for hospital room
navigation."""

    def __init__(self):
        """Initialize an empty graph."""
        self.adjacency_list = {}

    def add_room(self, room):
        """Add a room to the graph.

        Args:
            room: The room to be added to the graph.
        """
        if room not in self.adjacency_list:
            self.adjacency_list[room] = []

    def connect_rooms(self, room1, room2):
```

```
        """Connect two rooms in the graph.

        Args:
            room1: The first room.
            room2: The second room.
        """
        if room1 in self.adjacency_list and room2 in self.adjacency_list:
            self.adjacency_list[room1].append(room2)
            self.adjacency_list[room2].append(room1)

    def show_connections(self):
        """Show the connections between rooms in the graph."""
        for room, connections in self.adjacency_list.items():
            print(f"{room}: {', '.join(connections)}")
# Example usage:
if __name__ == "__main__":
    hospital_graph = Graph()
    hospital_graph.add_room("Ward A")
    hospital_graph.add_room("Ward B")
    hospital_graph.add_room("Room 101")

    hospital_graph.connect_rooms("Ward A", "Ward B")
    hospital_graph.connect_rooms("Ward A", "Room 101")

    hospital_graph.show_connections()  # Output: Ward A: Ward B, Room 101;
Ward B: Ward A; Room 101: Ward A
```

=== 10.1.py ===

['Patient A', 'Patient B', 'Patient C']

Patient A

['Patient B', 'Patient C']

=== 10.2.py ===

[(1, 'Patient B'), (2, 'Patient A'), (3, 'Patient C')]

Patient B

[(2, 'Patient A'), (3, 'Patient C')]

=== 10.3.py ===

{'name': 'John Doe', 'age': 30, 'condition': 'Flu'}

{'name': 'Jane Smith', 'age': 25, 'condition': 'Cold'}

None

=== 10.4.py ===

Scheduled Appointments (In Order): [('10:00 AM', 'Alice'), ('11:00 AM', 'Bob'), ('9:00 AM', 'Charlie')]

Scheduled Appointments after cancellation (In Order): [('11:00 AM', 'Bob'), ('9:00 AM', 'Charlie')]

=== 10.5.py ===

Ward A: Ward B, Room 101

Ward B: Ward A

Room 101: Ward A

Feature	**Data Structure**	**Justification**
Patient Check-In System	Queue	FIFO ensures patients are called in the exact order of arrival.
Emergency Case Handling	Priority Queue	Lower severity number = higher priority; ensures critical patients are treated first.
Medical Records Storage	Hash Table	O(1) access to patient records using Patient ID as key.
Doctor Appointment Scheduling	BST	Keeps appointments sorted by time for quick conflict detection and in-order view.
Hospital Room Navigation	Graph	Rooms/wards are vertices; physical connections are edges in an undirected graph.

Task Description #11: Smart City Traffic Control System

A city plans a Smart Traffic Management System that includes:

1. Traffic Signal Queue – Vehicles waiting at signals.

2. Emergency Vehicle Priority Handling – Ambulances and fire trucks

prioritized.

3. Vehicle Registration Lookup – Instant access to vehicle details.

4. Road Network Mapping – Roads and intersections connected

logically.

5. Parking Slot Availability – Track available and occupied slots.

Student Task

• For each feature, select the most appropriate data structure from

the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with

AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature

with comments and docstrings.

11.1

```
"""
Implement a Queue for a Traffic Signal System.
It should manage vehicles waiting at a red light and release them in order
(FIFO).
Include methods to add_vehicle, turn_light_green(release_one), and
waiting_count.
"""
class TrafficSignalQueue:
    def __init__(self):
        self.queue = []

    def add_vehicle(self, vehicle):
        """Add a vehicle to the queue."""
        self.queue.append(vehicle)

    def turn_light_green(self):
        """Release the next vehicle in the queue when the light turns
green."""
        if self.queue:
            return self.queue.pop(0)
        else:
            return "No vehicles waiting."
```

```
    def waiting_count(self):
        """Return the number of vehicles currently waiting at the red
light."""
        return len(self.queue)
# Example usage:
if __name__ == "__main__":
    traffic_queue = TrafficSignalQueue()
    traffic_queue.add_vehicle("Car 1")
    traffic_queue.add_vehicle("Car 2")
    traffic_queue.add_vehicle("Car 3")

    print("Vehicles waiting:", traffic_queue.waiting_count())  # Output:
Vehicles waiting: 3
    print(traffic_queue.turn_light_green())                    # Output: Car
1
    print("Vehicles waiting after light turns green:",
traffic_queue.waiting_count())  # Output: Vehicles waiting after light turns
green: 2
```

11.2

```
"""
Implement a Priority Queue for Traffic Emergency Handling.
It should prioritize ambulances and fire trucks over regular vehicles.
Include methods to add_vehicle(type, priority),
clear_path_for_highest_priority, and view_traffic.
"""
import heapq
class TrafficEmergencyQueue:
    def __init__(self):
        self.queue = []

    def add_vehicle(self, vehicle_type, priority):
        """Add a vehicle to the queue with a given priority."""
        heapq.heappush(self.queue, (priority, vehicle_type))

    def clear_path_for_highest_priority(self):
        """Clear the path for the vehicle with the highest priority."""
        if self.queue:
            return heapq.heappop(self.queue)[1]
        else:
            return "No vehicles to clear path for."

    def view_traffic(self):
        """View all vehicles currently in the queue sorted by priority."""
        return sorted(self.queue)
# Example usage:
```

```
if __name__ == "__main__":
    traffic_emergency_queue = TrafficEmergencyQueue()
    traffic_emergency_queue.add_vehicle("Ambulance", priority=1)
    traffic_emergency_queue.add_vehicle("Fire Truck", priority=2)
    traffic_emergency_queue.add_vehicle("Car", priority=3)

    print(traffic_emergency_queue.view_traffic())  # Output: [(1,
'Ambulance'), (2, 'Fire Truck'), (3, 'Car')]
    print(traffic_emergency_queue.clear_path_for_highest_priority())  #
Output: 'Ambulance'
    print(traffic_emergency_queue.view_traffic())  # Output: [(2, 'Fire
Truck'), (3, 'Car')]
```

11.3

```
"""
Implement a Hash Table for Vehicle Registration Lookup.
It should provide instant access to owner details using the license plate
number.
Include methods to register_vehicle, lookup_owner, and update_details.
"""
class VehicleRegistration:
    """A simple implementation of a vehicle registration lookup using a hash
table."""

    def __init__(self):
        """Initialize the vehicle registration lookup."""
        self.registrations = {}

    def register_vehicle(self, license_plate, owner_details):
        """Register a vehicle with its license plate and owner details.

        Args:
            license_plate: The license plate number of the vehicle.
            owner_details: A dictionary containing the owner's details (e.g.,
name, address).
        """
        self.registrations[license_plate] = owner_details

    def lookup_owner(self, license_plate):
        """Lookup the owner details using the license plate number.

        Args:
            license_plate: The license plate number to be looked up.

        Returns:
            The owner details associated with the license plate if found,
otherwise None.
```

```
        """
        return self.registrations.get(license_plate)

    def update_details(self, license_plate, new_owner_details):
        """Update the owner details for a given license plate number.

        Args:
            license_plate: The license plate number for which to update the
details.
            new_owner_details: A dictionary containing the new owner's
details.

        Raises:
            KeyError: If the license plate number is not found in the
registrations.
        """
        if license_plate not in self.registrations:
            raise KeyError("License plate not found in registrations")
        self.registrations[license_plate] = new_owner_details
# Example usage:
if __name__ == "__main__":
    vehicle_registry = VehicleRegistration()
    vehicle_registry.register_vehicle("ABC123", {"name": "John Doe",
"address": "123 Main St"})

    print(vehicle_registry.lookup_owner("ABC123"))  # Output: {'name': 'John
Doe', 'address': '123 Main St'}

    vehicle_registry.update_details("ABC123", {"name": "Jane Doe", "address":
"456 Elm St"})
    print(vehicle_registry.lookup_owner("ABC123"))  # Output: {'name': 'Jane
Doe', 'address': '456 Elm St'}
    print(vehicle_registry.lookup_owner("XYZ789"))  # Output: None
```

11.4

```
"""
Implement a Graph for Road Network Mapping.
It should connect roads and city intersections logically.
Include methods to add_intersection, add_road, and display_network.
"""
class Graph:
    """A simple implementation of a graph for road network mapping."""

    def __init__(self):
        """Initialize an empty graph."""
        self.adjacency_list = {}
```

```
    def add_intersection(self, intersection):
        """Add an intersection to the graph.

        Args:
            intersection: The intersection to be added to the graph.
        """
        if intersection not in self.adjacency_list:
            self.adjacency_list[intersection] = []

    def add_road(self, intersection1, intersection2):
        """Connect two intersections with a road in the graph.

        Args:
            intersection1: The first intersection.
            intersection2: The second intersection.
        """
        if intersection1 in self.adjacency_list and intersection2 in
self.adjacency_list:
            self.adjacency_list[intersection1].append(intersection2)
            self.adjacency_list[intersection2].append(intersection1)

    def display_network(self):
        """Display the connections between intersections in the graph."""
        for intersection, roads in self.adjacency_list.items():
            print(f"{intersection}: {', '.join(roads)}")
# Example usage:
if __name__ == "__main__":
    road_network = Graph()
    road_network.add_intersection("Intersection A")
    road_network.add_intersection("Intersection B")
    road_network.add_intersection("Intersection C")

    road_network.add_road("Intersection A", "Intersection B")
    road_network.add_road("Intersection A", "Intersection C")

    road_network.display_network()  # Output: Intersection A: Intersection B,
Intersection C; Intersection B: Intersection A; Intersection C: Intersection A
```

11.5

```
"""
Implement a Hash Table for Parking Slot Availability.
It should track available and occupied parking slots using the slot number as
the key.
Include methods to park_vehicle, free_slot, and check_availability.
"""
class ParkingLot:
```

```
    """A simple implementation of a parking lot using a hash table to track
slot availability."""

    def __init__(self, total_slots):
        """Initialize the parking lot with a specified number of slots.

        Args:
            total_slots: The total number of parking slots in the lot.
        """
        self.total_slots = total_slots
        self.slots = {slot_number: None for slot_number in range(1,
total_slots + 1)}

    def park_vehicle(self, slot_number, vehicle_details):
        """Park a vehicle in a specified slot.

        Args:
            slot_number: The slot number where the vehicle is to be parked.
            vehicle_details: A dictionary containing details about the vehicle
(e.g., license plate, owner).

        Raises:
            ValueError: If the slot number is invalid or already occupied.
        """
        if slot_number < 1 or slot_number > self.total_slots:
            raise ValueError("Invalid slot number")
        if self.slots[slot_number] is not None:
            raise ValueError("Slot is already occupied")
        self.slots[slot_number] = vehicle_details

    def free_slot(self, slot_number):
        """Free a specified parking slot.

        Args:
            slot_number: The slot number to be freed.

        Raises:
            ValueError: If the slot number is invalid or already free.
        """
        if slot_number < 1 or slot_number > self.total_slots:
            raise ValueError("Invalid slot number")
        if self.slots[slot_number] is None:
            raise ValueError("Slot is already free")
        self.slots[slot_number] = None

    def check_availability(self, slot_number):
        """Check if a specified parking slot is available.
```

```
        Args:
            slot_number: The slot number to check for availability.

        Returns:
            True if the slot is available, False otherwise.

        Raises:
            ValueError: If the slot number is invalid.
        """
        if slot_number < 1 or slot_number > self.total_slots:
            raise ValueError("Invalid slot number")
        return self.slots[slot_number] is None
# Example usage:
if __name__ == "__main__":
    parking_lot = ParkingLot(5)
    parking_lot.park_vehicle(1, {"license_plate": "ABC123", "owner": "John
Doe"})

    print(parking_lot.check_availability(1))  # Output: False
    print(parking_lot.check_availability(2))  # Output: True

    parking_lot.free_slot(1)
    print(parking_lot.check_availability(1))  # Output: True
```

=== 11.1.py ===

Vehicles waiting: 3

Car 1

Vehicles waiting after light turns green: 2

=== 11.2.py ===

[(1, 'Ambulance'), (2, 'Fire Truck'), (3, 'Car')]

Ambulance

[(2, 'Fire Truck'), (3, 'Car')]

=== 11.3.py ===

{'name': 'John Doe', 'address': '123 Main St'}

{'name': 'Jane Doe', 'address': '456 Elm St'}

None

=== 11.4.py ===

Intersection A: Intersection B, Intersection C

Intersection B: Intersection A

Intersection C: Intersection A

=== 11.5.py ===

False

True

True

Feature	**Data Structure**	**Justification**
Traffic Signal Queue	Queue	Vehicles are released in FIFO order when the light turns green.
Emergency Vehicle Priority	Priority Queue	Ambulances/fire trucks get lower priority numbers so they are cleared first.
Vehicle Registration Lookup	Hash Table	License plate as key gives O(1) lookup of owner details.
Road Network Mapping	Graph	Intersections are vertices; roads are edges connecting them.
Parking Slot Availability	Hash Table	Slot number as key gives instant O(1) check/update of occupancy status.

Task Description #12 & 13: Smart E-Commerce Platform – Data Structure

Challenge

An e-commerce company wants to build a Smart Online Shopping System

with:

1. Shopping Cart Management – Add and remove products

dynamically.

2. Order Processing System – Orders processed in the order they are

placed.

3. Top-Selling Products Tracker – Products ranked by sales count.

4. Product Search Engine – Fast lookup of products using product ID.

5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task:

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings.

12.1

```
"""
Implement a Singly Linked List for Shopping Cart Management.
It should allow dynamic adding and removing of products.
Include methods to add_item, remove_item, and view_cart.
"""
class Node:
    """A node in a singly linked list representing an item in the shopping cart."""

    def __init__(self, item):
        """Initialize a node with the given item and a reference to the next node.

        Args:
```

```
            item: The item stored in the node.
        """
        self.item = item
        self.next = None
class ShoppingCart:
    """A simple implementation of a shopping cart using a singly linked
list."""

    def __init__(self):
        """Initialize an empty shopping cart."""
        self.head = None

    def add_item(self, item):
        """Add an item to the shopping cart.

        Args:
            item: The item to be added to the cart.
        """
        new_node = Node(item)
        if self.head is None:
            self.head = new_node
            return
        last_node = self.head
        while last_node.next:
            last_node = last_node.next
        last_node.next = new_node

    def remove_item(self, item):
        """Remove an item from the shopping cart.

        Args:
            item: The item to be removed from the cart.
        """
        current_node = self.head
        previous_node = None
        while current_node:
            if current_node.item == item:
                if previous_node is None:
                    self.head = current_node.next
                else:
                    previous_node.next = current_node.next
                return
            previous_node = current_node
            current_node = current_node.next

    def view_cart(self):
        """View the contents of the shopping cart."""
        items = []
```

```
        current_node = self.head
        while current_node:
            items.append(current_node.item)
            current_node = current_node.next
        return items
# Example usage:
if __name__ == "__main__":
    cart = ShoppingCart()
    cart.add_item("Apple")
    cart.add_item("Banana")
    cart.add_item("Orange")

    print("Cart contents:", cart.view_cart())  # Output: Cart contents:
['Apple', 'Banana', 'Orange']

    cart.remove_item("Banana")
    print("Cart contents after removing Banana:", cart.view_cart())  # Output:
Cart contents after removing Banana: ['Apple', 'Orange']
```

12.2

```
"""
Implement a Queue for an Order Processing System.
It should process customer orders strictly in the order they are placed.
Include methods to place_order, process_next_order, and view_pending_orders.
"""
class Queue:
    """A simple implementation of a queue data structure for order
processing."""

    def __init__(self):
        """Initialize an empty queue."""
        self.orders = []

    def place_order(self, order):
        """Place a new order in the queue.

        Args:
            order: The order to be placed in the queue.
        """
        self.orders.append(order)

    def process_next_order(self):
        """Process and return the next order in the queue.

        Returns:
            The next order in the queue.
```

```
        Raises:
            IndexError: If there are no pending orders to process.
        """
        if self.is_empty():
            raise IndexError("No pending orders to process")
        return self.orders.pop(0)

    def view_pending_orders(self):
        """View the list of pending orders in the queue."""
        return self.orders

    def is_empty(self):
        """Check if there are no pending orders in the queue.

        Returns:
            True if there are no pending orders, False otherwise.
        """
        return len(self.orders) == 0
# Example usage:
if __name__ == "__main__":
    order_queue = Queue()
    order_queue.place_order("Order 1: 2x Widget A")
    order_queue.place_order("Order 2: 1x Widget B")
    order_queue.place_order("Order 3: 5x Widget C")

    print("Pending Orders:", order_queue.view_pending_orders())  # Output:
Pending Orders: ['Order 1: 2x Widget A', 'Order 2: 1x Widget B', 'Order 3: 5x
Widget C']

    print("Processing:", order_queue.process_next_order())  # Output:
Processing: Order 1: 2x Widget A
    print("Pending Orders after processing one order:",
order_queue.view_pending_orders())  # Output: Pending Orders after processing
one order: ['Order 2: 1x Widget B', 'Order 3: 5x Widget C']
    print("Processing:", order_queue.process_next_order())  # Output:
Processing: Order 2: 1x Widget B
    print("Pending Orders after processing another order:",
order_queue.view_pending_orders())  # Output
```

12.3

```
"""
Implement a Priority Queue for a Top-Selling Products Tracker.
It should rank products based on their total sales count.
Include methods to update_sales(product, count), get_top_seller, and
display_rankings.
"""
```

```
import heapq
class Product:
    """A class representing a product with its name and total sales count."""

    def __init__(self, name):
        """Initialize a product with its name and a sales count of zero.

        Args:
            name: The name of the product.
        """
        self.name = name
        self.sales_count = 0

    def update_sales(self, count):
        """Update the sales count for the product.

        Args:
            count: The number of sales to be added to the current sales count.
        """
        self.sales_count += count

    def __lt__(self, other):
        """Define less-than comparison based on sales count for priority
queue.

        Args:
            other: Another product to compare against.

        Returns:
            True if this product has fewer sales than the other, False
otherwise.
        """
        return self.sales_count < other.sales_count
class TopSellingProductsTracker:
    """A class to track top-selling products using a priority queue."""

    def __init__(self):
        """Initialize an empty priority queue for products."""
        self.products = []

    def update_sales(self, product_name, count):
        """Update the sales count for a product and maintain the priority
queue.

        Args:
            product_name: The name of the product to update.
            count: The number of sales to be added to the product's sales
count.
```

```
        """
        # Check if the product already exists in the priority queue
        for product in self.products:
            if product.name == product_name:
                # Update sales count and re-heapify the priority queue
                product.update_sales(count)
                heapq.heapify(self.products)
                return
        # If the product does not exist, create a new product and add it to
the queue
        new_product = Product(product_name)
        new_product.update_sales(count)
        heapq.heappush(self.products, new_product)

    def get_top_seller(self):
        """Get the top-selling product.

        Returns:
            The name of the top-selling product.

        Raises:
            IndexError: If there are no products in the tracker.
        """
        if not self.products:
            raise IndexError("No products in the tracker")
        return self.products[-1].name  # The top seller is at the end of the
min-heap

    def display_rankings(self):
        """Display the rankings of products based on their sales counts."""
        sorted_products = sorted(self.products, key=lambda p: p.sales_count,
reverse=True)
        for rank, product in enumerate(sorted_products, start=1):
            print(f"Rank {rank}: {product.name} with {product.sales_count}
sales")
# Example usage:
if __name__ == "__main__":
    tracker = TopSellingProductsTracker()
    tracker.update_sales("Product A", 100)
    tracker.update_sales("Product B", 150)
    tracker.update_sales("Product C", 120)

    print("Top Seller:", tracker.get_top_seller())  # Output: Product B
    print("Rankings:")
    tracker.display_rankings()  # Output: Rank 1: Product B with 150 sales;
Rank 2: Product C with 120 sales; Rank 3: Product A with 100 sales
```

12.4

```
"""
Implement a Hash Table for a Product Search Engine.
It should allow fast lookup of product details and prices using the Product
ID.
Include methods to add_product, search_product, and remove_product.
"""
class ProductSearchEngine:
    """A simple implementation of a product search engine using a hash
table."""

    def __init__(self):
        """Initialize the product search engine."""
        self.products = {}

    def add_product(self, product_id, product_details):
        """Add a product to the search engine.

        Args:
            product_id: The unique identifier for the product.
            product_details: A dictionary containing details about the product
(e.g., name, price).
        """
        self.products[product_id] = product_details

    def search_product(self, product_id):
        """Search for a product using its Product ID.

        Args:
            product_id: The unique identifier for the product to be searched.

        Returns:
            The product details associated with the Product ID if found,
otherwise None.
        """
        return self.products.get(product_id)

    def remove_product(self, product_id):
        """Remove a product from the search engine.

        Args:
            product_id: The unique identifier for the product to be removed.

        Raises:
            KeyError: If the Product ID is not found in the products.
        """
        if product_id not in self.products:
            raise KeyError("Product ID not found in products")
        del self.products[product_id]
```

```
# Example usage:
if __name__ == "__main__":
    search_engine = ProductSearchEngine()
    search_engine.add_product("P001", {"name": "Laptop", "price": 999.99})

    print(search_engine.search_product("P001"))  # Output: {'name': 'Laptop',
'price': 999.99}

    search_engine.remove_product("P001")
    print(search_engine.search_product("P001"))  # Output: None
```

12.5

```
"""
Implement a Graph for Delivery Route Planning.
It should connect warehouses to delivery locations to map out possible routes.
Include methods to add_location, add_route, and show_delivery_map.
"""
class Graph:
    """A simple implementation of a graph for delivery route planning."""

    def __init__(self):
        """Initialize an empty graph."""
        self.adjacency_list = {}

    def add_location(self, location):
        """Add a location to the graph.

        Args:
            location: The location to be added to the graph.
        """
        if location not in self.adjacency_list:
            self.adjacency_list[location] = []

    def add_route(self, location1, location2):
        """Connect two locations with a route in the graph.

        Args:
            location1: The first location.
            location2: The second location.
        """
        if location1 in self.adjacency_list and location2 in
self.adjacency_list:
            self.adjacency_list[location1].append(location2)
            self.adjacency_list[location2].append(location1)

    def show_delivery_map(self):
```

```
        """Show the connections between locations in the graph."""
        for location, routes in self.adjacency_list.items():
            print(f"{location}: {', '.join(routes)}")
# Example usage:
if __name__ == "__main__":
    delivery_map = Graph()
    delivery_map.add_location("Warehouse A")
    delivery_map.add_location("Warehouse B")
    delivery_map.add_location("Delivery Location 1")

    delivery_map.add_route("Warehouse A", "Warehouse B")
    delivery_map.add_route("Warehouse A", "Delivery Location 1")

    delivery_map.show_delivery_map()  # Output: Warehouse A: Warehouse B,
Delivery Location 1; Warehouse B: Warehouse A; Delivery Location 1: Warehouse
A
```

=== 12.1.py ===

Cart contents: ['Apple', 'Banana', 'Orange']

Cart contents after removing Banana: ['Apple', 'Orange']

=== 12.2.py ===

Pending Orders: ['Order 1: 2x Widget A', 'Order 2: 1x Widget B', 'Order 3: 5x Widget C']

Processing: Order 1: 2x Widget A

Pending Orders after processing one order: ['Order 2: 1x Widget B', 'Order 3: 5x Widget C']

Processing: Order 2: 1x Widget B

Pending Orders after processing another order: ['Order 3: 5x Widget C']

=== 12.3.py ===

Top Seller: Product C

Rankings:

Rank 1: Product B with 150 sales

Rank 2: Product C with 120 sales

Rank 3: Product A with 100 sales

=== 12.4.py ===

{'name': 'Laptop', 'price': 999.99}

None

=== 12.5.py ===

Warehouse A: Warehouse B, Delivery Location 1

Warehouse B: Warehouse A

Delivery Location 1: Warehouse A

Feature	Data Structure	Justification
Shopping Cart Management	Linked List	Dynamic insertion and removal of products without fixed size.
Order Processing System	Queue	Orders are processed in the exact sequence they were placed (FIFO).
Top-Selling Products Tracker	Priority Queue	Min-heap ranks products; top seller always retrievable efficiently.
Product Search Engine	Hash Table	Product ID as key gives O(1) add, search, and delete.
Delivery Route Planning	Graph	Warehouses and delivery points are vertices; possible routes are edges

Task Description #14: Smart Banking Management System – Data Structure Challenge

A bank plans to develop a Smart Banking System that includes:

1. Customer Service Token System – Customers are served in order of arrival.
2. Loan Approval Priority Handling – High-value or urgent loans processed first.
3. Account Information Lookup – Fast retrieval of customer details using Account Number.
4. Transaction History Management – Maintain chronological transaction records.
5. ATM Network Connectivity – Represent connections between ATMs across cities.

Student Task:

• For each feature, select the most appropriate data structure from

the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program

with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature

with comments and docstrings.

14.1

```
"""
Implement a Queue for a Customer Service Token System.
It should serve banking customers in the order they pulled their token.
Include methods to issue_token, call_next_token, and waiting_tokens.
"""
class Queue:
    """A simple implementation of a queue data structure for a customer
service token system."""

    def __init__(self):
        """Initialize an empty queue."""
        self.tokens = []

    def issue_token(self, token):
        """Issue a new token to a customer.

        Args:
            token: The token to be issued to the customer.
        """
        self.tokens.append(token)
```

```
    def call_next_token(self):
        """Call and return the next token in the queue.

        Returns:
            The next token in the queue.

        Raises:
            IndexError: If there are no waiting tokens to call.
        """
        if self.is_empty():
            raise IndexError("No waiting tokens to call")
        return self.tokens.pop(0)

    def waiting_tokens(self):
        """View the list of waiting tokens in the queue."""
        return self.tokens

    def is_empty(self):
        """Check if there are no waiting tokens in the queue.

        Returns:
            True if there are no waiting tokens, False otherwise.
        """
        return len(self.tokens) == 0
# Example usage:
if __name__ == "__main__":
    token_queue = Queue()
    token_queue.issue_token("Token 1: Customer A")
    token_queue.issue_token("Token 2: Customer B")
    token_queue.issue_token("Token 3: Customer C")

    print("Waiting Tokens:", token_queue.waiting_tokens())  # Output: Waiting
Tokens: ['Token 1: Customer A', 'Token 2: Customer B', 'Token 3: Customer C']

    print("Calling:", token_queue.call_next_token())  # Output: Calling: Token
1: Customer A
    print("Waiting Tokens after calling one token:",
token_queue.waiting_tokens())  # Output: Waiting Tokens after calling one
token: ['Token 2: Customer B', 'Token 3: Customer C']
    print("Calling:", token_queue.call_next_token())  # Output: Calling: Token
2: Customer B
    print("Waiting Tokens after calling another token:",
token_queue.waiting_tokens())  # Output: Waiting Tokens after calling another
token: ['Token 3: Customer C']
```

14.2

```
"""
Implement a Priority Queue for Loan Approval Handling.
It should process high-value or urgent loan applications first.
Include methods to submit_application(name, priority),
process_highest_priority, and view_queue.
"""
import heapq
class LoanApplication:
    """A class representing a loan application with applicant's name and
priority."""

    def __init__(self, name, priority):
        """Initialize a loan application with the applicant's name and
priority.

        Args:
            name: The name of the applicant.
            priority: The priority of the loan application (lower values
indicate higher priority).
        """
        self.name = name
        self.priority = priority

    def __lt__(self, other):
        """Define less-than comparison based on priority for the priority
queue.

        Args:
            other: Another loan application to compare against.

        Returns:
            True if this application has higher priority than the other, False
otherwise.
        """
        return self.priority < other.priority
class LoanApprovalHandler:
    """A class to handle loan approvals using a priority queue."""

    def __init__(self):
        """Initialize an empty priority queue for loan applications."""
        self.applications = []

    def submit_application(self, name, priority):
        """Submit a new loan application to the priority queue.

        Args:
            name: The name of the applicant.
```

```
            priority: The priority of the loan application (lower values
indicate higher priority).
        """
        application = LoanApplication(name, priority)
        heapq.heappush(self.applications, application)

    def process_highest_priority(self):
        """Process and return the highest priority loan application.

        Returns:
            The name of the applicant with the highest priority.

        Raises:
            IndexError: If there are no pending applications to process.
        """
        if self.is_empty():
            raise IndexError("No pending applications to process")
        return heapq.heappop(self.applications).name

    def view_queue(self):
        """View the list of pending loan applications in the priority
queue."""
        return [(app.name, app.priority) for app in sorted(self.applications)]

    def is_empty(self):
        """Check if there are no pending loan applications in the queue.

        Returns:
            True if there are no pending applications, False otherwise.
        """
        return len(self.applications) == 0
# Example usage:
if __name__ == "__main__":
    handler = LoanApprovalHandler()
    handler.submit_application("Alice", priority=2)
    handler.submit_application("Bob", priority=1)
    handler.submit_application("Charlie", priority=3)

    print("Pending Applications:", handler.view_queue())  # Output: Pending
Applications: [('Bob', 1), ('Alice', 2), ('Charlie', 3)]

    print("Processing:", handler.process_highest_priority())  # Output:
Processing: Bob
    print("Pending Applications after processing one application:",
handler.view_queue())  # Output: Pending Applications after processing one
application: [('Alice', 2), ('Charlie', 3)]
    print("Processing:", handler.process_highest_priority())  # Output:
Processing: Alice
```

```
    print("Pending Applications after processing another application:",
handler.view_queue())  # Output: Pending Applications after processing another
application: [('Charlie', 3)]
```

14.3

```
"""
Implement a Hash Table for Account Information Lookup.
It should retrieve customer balances and details quickly using the Account
Number.
Include methods to open_account, get_account_details, and close_account.
"""
class AccountInformation:
    """A simple implementation of an account information lookup using a hash
table."""

    def __init__(self):
        """Initialize the account information lookup."""
        self.accounts = {}

    def open_account(self, account_number, account_details):
        """Open a new account with the given account number and details.

        Args:
            account_number: The unique identifier for the account.
            account_details: A dictionary containing details about the account
(e.g., customer name, balance).
        """
        self.accounts[account_number] = account_details

    def get_account_details(self, account_number):
        """Retrieve the account details using the account number.

        Args:
            account_number: The unique identifier for the account to be
retrieved.

        Returns:
            The account details associated with the account number if found,
otherwise None.
        """
        return self.accounts.get(account_number)

    def close_account(self, account_number):
        """Close an existing account by removing it from the accounts.

        Args:
```

```
            account_number: The unique identifier for the account to be
closed.

        Raises:
            KeyError: If the account number is not found in the accounts.
        """
        if account_number not in self.accounts:
            raise KeyError("Account number not found in accounts")
        del self.accounts[account_number]
# Example usage:
if __name__ == "__main__":
    account_info = AccountInformation()
    account_info.open_account("ACC123", {"customer_name": "Alice Smith",
"balance": 1500.00})
    print(account_info.get_account_details("ACC123"))  # Output:
{'customer_name': 'Alice Smith', 'balance': 1500.00}
    account_info.close_account("ACC123")
    print(account_info.get_account_details("ACC123"))  # Output: None
```

14.4

```
"""
Implement a Stack for Transaction History Management.
It should maintain records chronologically, showing the most recent
transaction first (LIFO).
Include methods to add_transaction, view_latest_transaction, and
print_history.
"""
class Stack:
    """A simple implementation of a stack for transaction history
management."""

    def __init__(self):
        """Initialize an empty stack."""
        self.stack = []

    def add_transaction(self, transaction):
        """Add a transaction to the stack.

        Args:
            transaction: The transaction to be added to the stack.
        """
        self.stack.append(transaction)

    def view_latest_transaction(self):
        """View the most recent transaction in the stack."""
        if self.stack:
```

```
            return self.stack[-1]
        return None

    def print_history(self):
        """Print the transaction history in LIFO order."""
        for transaction in reversed(self.stack):
            print(transaction)
# Example usage:
if __name__ == "__main__":
    transaction_history = Stack()
    transaction_history.add_transaction("Transaction 1: $100")
    transaction_history.add_transaction("Transaction 2: $200")
    transaction_history.add_transaction("Transaction 3: $300")

    print("Latest transaction:",
transaction_history.view_latest_transaction())  # Output: Latest transaction:
Transaction 3: $300
    print("Transaction history:")
    transaction_history.print_history()  # Output: Transaction 3: $300;
Transaction 2: $200; Transaction 1: $100
```

14.5

```
"""
Implement a Graph for ATM Network Connectivity.
It should represent the data connections between different ATMs across cities.
Include methods to add_atm, connect_atms, and display_network.
"""
class Graph:
    """A simple implementation of a graph for ATM network connectivity."""

    def __init__(self):
        """Initialize an empty graph."""
        self.adjacency_list = {}

    def add_atm(self, atm):
        """Add an ATM to the graph.

        Args:
            atm: The ATM to be added to the graph.
        """
        if atm not in self.adjacency_list:
            self.adjacency_list[atm] = []

    def connect_atms(self, atm1, atm2):
        """Connect two ATMs in the graph.

        Args:
```

```
            atm1: The first ATM.
            atm2: The second ATM.
        """
        if atm1 in self.adjacency_list and atm2 in self.adjacency_list:
            self.adjacency_list[atm1].append(atm2)
            self.adjacency_list[atm2].append(atm1)

    def display_network(self):
        """Display the connections between ATMs in the graph."""
        for atm, connections in self.adjacency_list.items():
            print(f"{atm}: {', '.join(connections)}")
# Example usage:
if __name__ == "__main__":
    atm_network = Graph()
    atm_network.add_atm("ATM A")
    atm_network.add_atm("ATM B")
    atm_network.add_atm("ATM C")

    atm_network.connect_atms("ATM A", "ATM B")
    atm_network.connect_atms("ATM A", "ATM C")

    atm_network.display_network()  # Output: ATM A: ATM B, ATM C; ATM B: ATM
A; ATM C: ATM A
```

=== 14.1.py ===

Waiting Tokens: ['Token 1: Customer A', 'Token 2: Customer B', 'Token 3: Customer C']

Calling: Token 1: Customer A

Waiting Tokens after calling one token: ['Token 2: Customer B', 'Token 3: Customer C']

Calling: Token 2: Customer B

Waiting Tokens after calling another token: ['Token 3: Customer C']

=== 14.2.py ===

Pending Applications: [('Bob', 1), ('Alice', 2), ('Charlie', 3)]

Processing: Bob

Pending Applications after processing one application: [('Alice', 2), ('Charlie', 3)]

Processing: Alice

Pending Applications after processing another application: [('Charlie', 3)]

=== 14.3.py ===

{'customer_name': 'Alice Smith', 'balance': 1500.0}

None

=== 14.4.py ===

Latest transaction: Transaction 3: $300

Transaction history:

Transaction 3: $300

Transaction 2: $200

Transaction 1: $100

=== 14.5.py ===

ATM A: ATM B, ATM C

ATM B: ATM A

ATM C: ATM A

Feature	**Data Structure**	**Justification**
Customer Service Token System	Queue	Customers are served in the order they pulled their token (FIFO).
Loan Approval Priority Handling	Priority Queue	Urgent/high-value loans get lower priority numbers and are processed first.
Account Information Lookup	Hash Table	Account number as key gives O(1) retrieval of customer details.
Transaction History Management	Stack	LIFO order naturally shows the most recent transaction first (undo-friendly).
ATM Network Connectivity	Graph	ATMs are vertices; data/physical connections between them are edges.

Task Description #15: Online Learning Management System – Data

Structure Selection

An online education platform wants to develop a Learning Management

System that handles:

1. Student Login Authentication – Quick validation of user credentials.

2. Assignment Submission Queue – Submissions processed in order of upload.

3. Top Performer Ranking – Rank students based on scores.

4. Course Prerequisite Structure – Represent subject dependencies.

5. Undo Feature in Online Editor – Reverse recent actions.

Student Task:

• Select the most appropriate data structure for each feature from the given list.

• Justify your selection in 2–3 sentences per feature.

• Implement one selected feature in Python using AI-assisted code generation.

Expected Output:

• Feature → Data Structure → Justification table.

• Functional Python program with proper documentation.

15.1

```
"""
Implement a Hash Table for Student Login Authentication.
It should quickly validate user credentials mapping usernames to passwords.
Include methods to register_user, authenticate_user, and delete_user.
"""
class StudentLogin:
    """A simple implementation of a student login authentication system using
a hash table."""

    def __init__(self):
        """Initialize the student login authentication system."""
        self.users = {}

    def register_user(self, username, password):
        """Register a new user with a username and password.

        Args:
            username: The unique identifier for the user.
            password: The password associated with the username.
```

```
        Raises:
            ValueError: If the username is already taken.
        """
        if username in self.users:
            raise ValueError("Username already taken")
        self.users[username] = password

    def authenticate_user(self, username, password):
        """Authenticate a user by validating their credentials.

        Args:
            username: The username of the user to be authenticated.
            password: The password provided for authentication.

        Returns:
            True if the credentials are valid, False otherwise.
        """
        return self.users.get(username) == password

    def delete_user(self, username):
        """Delete an existing user from the authentication system.

        Args:
            username: The username of the user to be deleted.

        Raises:
            KeyError: If the username is not found in the users.
        """
        if username not in self.users:
            raise KeyError("Username not found in users")
        del self.users[username]
# Example usage:
if __name__ == "__main__":
    login_system = StudentLogin()
    login_system.register_user("student1", "password123")

    print(login_system.authenticate_user("student1", "password123"))  #
Output: True
    print(login_system.authenticate_user("student1", "wrongpassword"))  #
Output: False

    login_system.delete_user("student1")
    print(login_system.authenticate_user("student1", "password123"))  #
Output: False
    print(login_system.authenticate_user("student2", "password123"))  #
Output: False
```

15.2

```
"""
Implement a Queue for an Assignment Submission System.
It should process grading in the order students uploaded their files.
Include methods to submit_assignment, grade_next, and view_pending_grading.
"""
class Queue:
    """A simple implementation of a queue data structure for an assignment
submission system."""

    def __init__(self):
        """Initialize an empty queue."""
        self.submissions = []

    def submit_assignment(self, submission):
        """Submit a new assignment to the queue.

        Args:
            submission: The assignment submission to be added to the queue.
        """
        self.submissions.append(submission)

    def grade_next(self):
        """Grade and return the next assignment in the queue.

        Returns:
            The next assignment submission in the queue.

        Raises:
            IndexError: If there are no pending submissions to grade.
        """
        if self.is_empty():
            raise IndexError("No pending submissions to grade")
        return self.submissions.pop(0)

    def view_pending_grading(self):
        """View the list of pending submissions in the queue."""
        return self.submissions

    def is_empty(self):
        """Check if there are no pending submissions in the queue.

        Returns:
            True if there are no pending submissions, False otherwise.
        """
        return len(self.submissions) == 0
# Example usage:
if __name__ == "__main__":
    submission_queue = Queue()
```

```
    submission_queue.submit_assignment("Submission 1: Student A")
    submission_queue.submit_assignment("Submission 2: Student B")
    submission_queue.submit_assignment("Submission 3: Student C")

    print("Pending Submissions:", submission_queue.view_pending_grading())  #
Output: Pending Submissions: ['Submission 1: Student A', 'Submission 2:
Student B', 'Submission 3: Student C']

    print("Grading:", submission_queue.grade_next())  # Output: Grading:
Submission 1: Student A
    print("Pending Submissions after grading one assignment:",
submission_queue.view_pending_grading())  # Output: Pending Submissions after
grading one assignment: ['Submission 2: Student B', 'Submission 3: Student C']
    print("Grading:", submission_queue.grade_next())  # Output: Grading:
Submission 2: Student B
    print("Pending Submissions after grading another assignment:",
submission_queue.view_pending_grading())  # Output: Pending Submissions after
grading another assignment: ['Submission 3: Student C']
```

15.3

```
"""
Implement a Priority Queue for a Top Performer Ranking system.
It should keep students ranked based on their highest scores.
Include methods to add_score(student, score), get_top_student, and
view_leaderboard.
"""
import heapq
class Student:
    """A class representing a student with their name and highest score."""

    def __init__(self, name):
        """Initialize a student with their name and a highest score of zero.

        Args:
            name: The name of the student.
        """
        self.name = name
        self.highest_score = 0

    def add_score(self, score):
        """Update the highest score for the student if the new score is
higher.

        Args:
            score: The new score to be compared with the current highest
score.
        """
```

```
        if score > self.highest_score:
            self.highest_score = score

    def __lt__(self, other):
        """Define less-than comparison based on highest score for priority
queue.

        Args:
            other: Another student to compare against.

        Returns:
            True if this student has a lower highest score than the other,
False otherwise.
        """
        return self.highest_score < other.highest_score
class TopPerformerRanking:
    """A class to track top performers using a priority queue."""

    def __init__(self):
        """Initialize an empty priority queue for students."""
        self.students = []

    def add_score(self, student_name, score):
        """Add a score for a student and maintain the priority queue.

        Args:
            student_name: The name of the student to update.
            score: The new score to be added for the student.
        """
        # Check if the student already exists in the priority queue
        for student in self.students:
            if student.name == student_name:
                # Update highest score and re-heapify the priority queue
                student.add_score(score)
                heapq.heapify(self.students)
                return
        # If the student does not exist, create a new student and add it to
the queue
        new_student = Student(student_name)
        new_student.add_score(score)
        heapq.heappush(self.students, new_student)

    def get_top_student(self):
        """Get the name of the top-performing student.

        Returns:
            The name of the student with the highest score.
```

```
        Raises:
            IndexError: If there are no students in the ranking.
        """
        if self.is_empty():
            raise IndexError("No students in the ranking")
        return self.students[-1].name

    def view_leaderboard(self):
        """View the leaderboard of students ranked by their highest scores."""
        return [(student.name, student.highest_score) for student in
sorted(self.students, key=lambda s: s.highest_score, reverse=True)]

    def is_empty(self):
        """Check if there are no students in the ranking.

        Returns:
            True if there are no students, False otherwise.
        """
        return len(self.students) == 0
# Example usage:
if __name__ == "__main__":
    ranking = TopPerformerRanking()
    ranking.add_score("Alice", 85)
    ranking.add_score("Bob", 90)
    ranking.add_score("Charlie", 80)
    ranking.add_score("Alice", 95)  # Alice's highest score should now be 95

    print(ranking.get_top_student())  # Output: Alice
    print(ranking.view_leaderboard())  # Output: [('Alice', 95), ('Bob', 90),
('Charlie', 80)]
    ranking.add_score("Bob", 92)  # Bob's highest score should now be 92
    print(ranking.get_top_student())  # Output: Alice
    print(ranking.view_leaderboard())  # Output: [('Alice', 95), ('Bob', 92),
('Charlie', 80)]
```

15.4

```
"""
Implement a Directed Graph for a Course Prerequisite Structure.
It should show which subjects must be taken before others (e.g., Math 101 ->
Math 102).
Include methods to add_course, add_prerequisite, and view_course_path.
"""
class Graph:
    """A simple implementation of a directed graph for course prerequisite
structure."""

    def __init__(self):
```

```
        """Initialize an empty graph."""
        self.adjacency_list = {}

    def add_course(self, course):
        """Add a course to the graph.

        Args:
            course: The course to be added to the graph.
        """
        if course not in self.adjacency_list:
            self.adjacency_list[course] = []

    def add_prerequisite(self, course, prerequisite):
        """Add a prerequisite relationship between two courses in the graph.

        Args:
            course: The course that has a prerequisite.
            prerequisite: The course that is a prerequisite for the given
course.
        """
        if course in self.adjacency_list and prerequisite in
self.adjacency_list:
            self.adjacency_list[course].append(prerequisite)

    def view_course_path(self, course, visited=None):
        """View the path of prerequisites for a given course.

        Args:
            course: The course for which to view the prerequisite path.
            visited: A set of visited courses to avoid cycles.
        """
        if visited is None:
            visited = set()
        if course not in self.adjacency_list:
            return []
        if course in visited:
            return []

        visited.add(course)
        prerequisites = []
        for prereq in self.adjacency_list[course]:
            prerequisites.extend(self.view_course_path(prereq, visited))

        prerequisites.append(course)
        return prerequisites
# Example usage:
if __name__ == "__main__":
    course_graph = Graph()
```

```
    course_graph.add_course("Math 101")
    course_graph.add_course("Math 102")
    course_graph.add_course("Math 201")

    course_graph.add_prerequisite("Math 102", "Math 101")
    course_graph.add_prerequisite("Math 201", "Math 102")

    print(course_graph.view_course_path("Math 201"))  # Output: ['Math 101',
'Math 102', 'Math 201']
```

15.5

```
"""
Implement a Stack for an Undo Feature in an Online Editor.
It should save typing states to reverse recent actions one by one (LIFO).
Include methods to type_text(state), undo, and view_current_state.
"""
class Stack:
    """A simple implementation of a stack for an undo feature in an online
editor."""

    def __init__(self):
        """Initialize an empty stack."""
        self.stack = []

    def type_text(self, state):
        """Save the current typing state to the stack.

        Args:
            state: The current state of the text to be saved.
        """
        self.stack.append(state)

    def undo(self):
        """Undo the most recent action and return the previous state.

        Returns:
            The previous state after undoing the most recent action.

        Raises:
            IndexError: If there are no states to undo.
        """
        if not self.stack:
            raise IndexError("No actions to undo")
        return self.stack.pop()

    def view_current_state(self):
```

```
        """View the current state of the text."""
        if self.stack:
            return self.stack[-1]
        return None
# Example usage:
if __name__ == "__main__":
    editor_stack = Stack()
    editor_stack.type_text("Hello")
    editor_stack.type_text("Hello, World")
    editor_stack.type_text("Hello, World!")

    print("Current state:", editor_stack.view_current_state())  # Output:
Current state: Hello, World!
    print("Undoing last action...")
    editor_stack.undo()
    print("Current state after undo:", editor_stack.view_current_state())  #
Output: Current state after undo: Hello, World
    print("Undoing another action...")
    editor_stack.undo()
    print("Current state after another undo:",
editor_stack.view_current_state())  # Output: Current state after another
undo: Hello
```

=== 15.1.py ===

True

False

False

False

=== 15.2.py ===

Pending Submissions: ['Submission 1: Student A', 'Submission 2: Student B', 'Submission 3: Student C']

Grading: Submission 1: Student A

Pending Submissions after grading one assignment: ['Submission 2: Student B', 'Submission 3: Student C']

Grading: Submission 2: Student B

Pending Submissions after grading another assignment: ['Submission 3: Student C']

=== 15.3.py ===

Alice

[('Alice', 95), ('Bob', 90), ('Charlie', 80)]

Alice

[('Alice', 95), ('Bob', 92), ('Charlie', 80)]

=== 15.4.py ===

['Math 101', 'Math 102', 'Math 201']

=== 15.5.py ===

Current state: Hello, World!

Undoing last action...

Current state after undo: Hello, World

Undoing another action...

Current state after another undo: Hello

Feature	Data Structure	Justification
Student Login Authentication	Hash Table	Username as key maps directly to password for O(1) credential validation.
Assignment Submission Queue	Queue	Submissions are graded in the order they were uploaded (FIFO).
Top Performer Ranking	Priority Queue	Min-heap keeps students ranked; top scorer is efficiently retrievable.
Course Prerequisite Structure	Directed Graph	Directed edges represent "must complete before" dependency between courses.
Undo Feature in Online Editor	Stack	Each typing state is pushed; undo pops the last state (LIFO).